



第9章 近似算法



第9章 近似算法

迄今为止，所有的NP完全问题都还没有多项式时间算法。对于这类问题，通常可采取以下几种解题策略。

- (1) 只对问题的特殊实例求解
- (2) 用动态规划法或分支限界法求解
- (3) 用概率算法求解
- (4) 只求近似解
- (5) 用启发式方法求解

本章主要讨论解NP完全问题的**近似算法**。



9.1 近似算法的性能

若一个最优化问题的最优值为 c^* ，求解该问题的一个近似算法求得的近似最优解相应的目标函数值为 c ，则将该**近似算法的性能比**定义为 $\eta = \max \left\{ \frac{c}{c^*}, \frac{c^*}{c} \right\}$ 。在通常情况下，该性能比是问题输入规模 n 的一个函数 $\rho(n)$ (Ratio Bound)，即

$$\max \left\{ \frac{c}{c^*}, \frac{c^*}{c} \right\} \leq \rho(n)。$$

该**近似算法的相对误差**定义为 $\lambda = \left| \frac{c - c^*}{c^*} \right|$ 。若对问题的输入规模 n ，有一函数 $\varepsilon(n)$ 使得 $\left| \frac{c - c^*}{c^*} \right| \leq \varepsilon(n)$ ，则称 $\varepsilon(n)$ 为该**近似算法的相对误差界**。近似算法的性能比 $\rho(n)$ 与相对误差界 $\varepsilon(n)$ 之间显然有如下关系：
 $\varepsilon(n) \leq \rho(n) - 1。$



9.2 顶点覆盖问题的近似算法

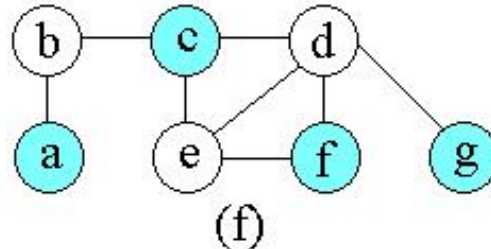
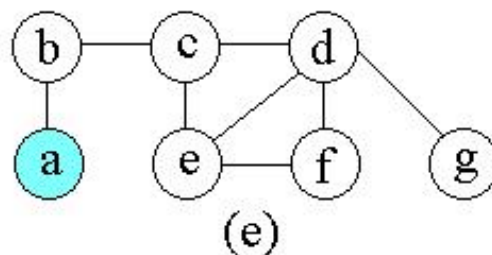
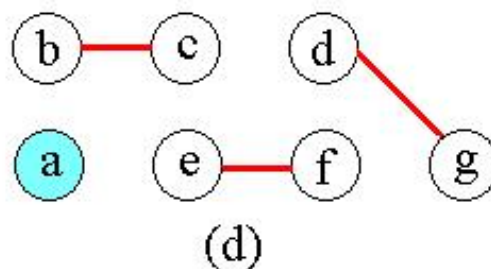
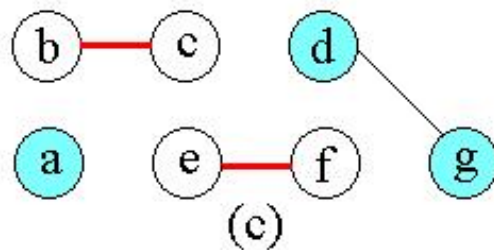
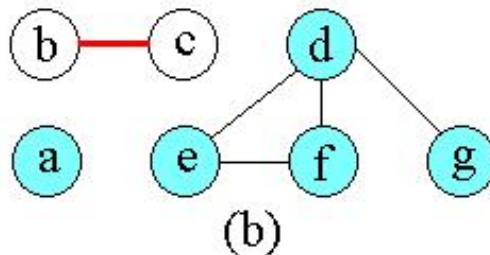
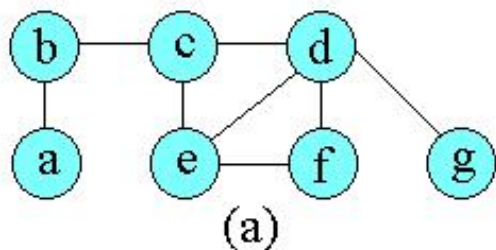
问题描述：无向图 $G=(V, E)$ 的顶点覆盖是它的顶点集 V 的一个子集 $V' \subseteq V$ ，使得若 (u, v) 是 G 的一条边，则 $v \in V'$ 或 $u \in V'$ 。顶点覆盖 V' 的大小是它所包含的顶点个数 $|V'|$ 。

```
VertexSet approxVertexCover ( Graph g )
{
    cset= $\emptyset$ ;
    e1=g.e;
    while (e1  $\neq \emptyset$ ) {
        从e1中任取一条边(u, v);
        cset=cset  $\cup$  {u, v};
        从e1中删去与u和v相关联的所有边;
    }
    return c
}
```

Cset用来存储顶点覆盖中的各顶点。初始为空，不断从边集e1中选取一边 (u, v) ，将边的端点加入cset中，并将e1中已被u和v覆盖的边删去，直至cset已覆盖所有边。即e1为空。



9.2 顶点覆盖问题的近似算法



图(a)~(e)说明了算法的运行过程及结果。(e)表示算法产生的近似最优顶点覆盖 $cset$,它由顶点 b, c, d, e, f, g 所组成。(f)是图G的一个最小顶点覆盖,它只含有3个顶点: b, d 和 e 。

算法approxVertexCover的性能比为2。



近似算法性能分析

- 时间复杂度为 $O(|E|)$
- Ratio Bound为2
 - 令 E' 为选中的边集, 若 $(u, v) \in E'$, 则与其相邻的边都被删除, 因此 E' 中无相邻边
 - 每次选一条边, 则每次又两个顶点加入解集 A ,
 $|A|=2|E'|$
 - 设 OPT 为最优解, OPT 必覆盖 E' , 由于 E' 中无邻接边, OPT 至少包含 E' 中每条边的一个顶点
 - $|E'| \leq |OPT|$, $|A|=2|E'| \leq 2|OPT|$, $|A|/|OPT| \leq 2$



9.3 旅行售货员问题近似算法

问题描述：给定一个完全无向图 $G=(V, E)$ ，其每一边 $(u, v) \in E$ 有一非负整数费用 $c(u, v)$ 。要找出 G 的最小费用哈密顿回路。

旅行售货员问题的一些特殊性质：

比如，费用函数 c 往往具有三角不等式性质，即对任意的3个顶点 $u, v, w \in V$ ，有： $c(u, w) \leq c(u, v) + c(v, w)$ 。当图 G 中的顶点就是平面上的点，任意2顶点间的费用就是这2点间的欧氏距离时，费用函数 c 就具有三角不等式性质。



9.3.1 具有三角不等式性质的旅行售货员问题

对于给定的无向图G，可以利用找图G的最小生成树的算法设计找近似最优的旅行售货员回路的算法。

```
void approxTSP (Graph g)
```

```
{
```

(1) 选择g的任一顶点r;

(2) 用Prim算法找出带权图g的一棵以r为根的最小生成树T;

(3) 前序遍历树T得到的顶点表L;

(4) 将r加到表L的末尾，按表L中顶点次序组成回路H，作为计算结果返回;

```
}
```

当费用函数满足三角不等式时，算法找出的旅行售货员回路的费用不会超过最优旅行售货员回路费用的2倍。



最小生成树

Prim算法

设 $G=(V, E)$ 是连通带权图， $V=\{1, 2, \dots, n\}$ 。

构造 G 的最小生成树的Prim算法的**基本思想**是：首先置 $S=\{1\}$ ，然后，只要 S 是 V 的真子集，就作如下的**贪心选择**：选取满足条件 $i \in S$ ， $j \in V-S$ ，且 $c[i][j]$ 最小的边，将顶点 j 添加到 S 中。这个过程一直进行到 $S=V$ 时为止。

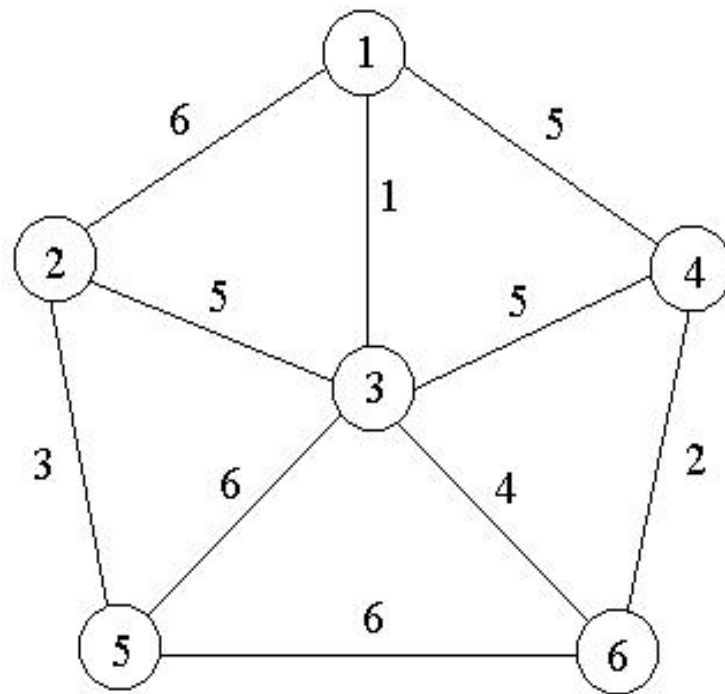
在这个过程中选取到的所有边恰好构成 G 的一棵**最小生成树**。



最小生成树

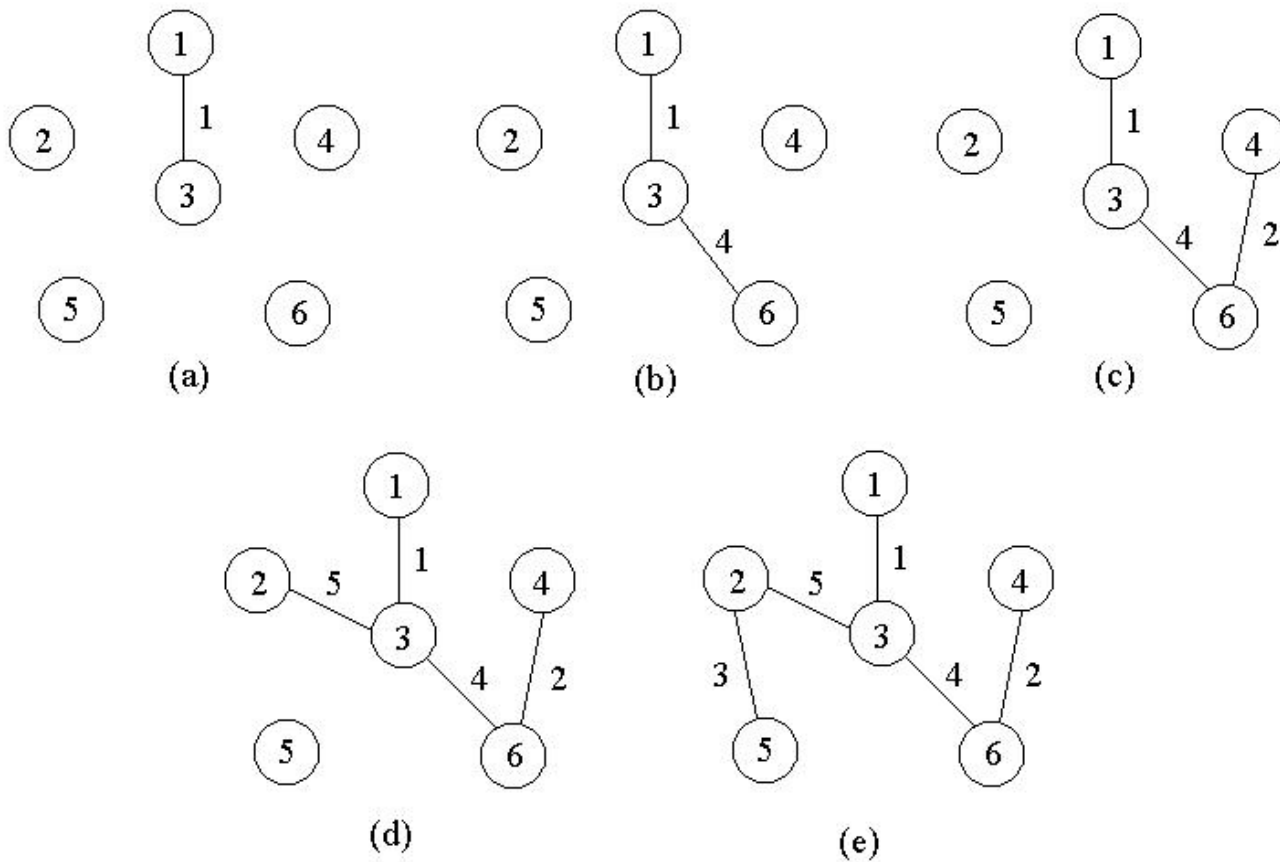
利用最小生成树性质和数学归纳法容易证明，上述算法中的**边集合T始终包含G的某棵最小生成树中的边**。因此，在算法结束时，T中的所有边构成G的一棵最小生成树。

例如，对于右图中的带权图，按**Prim算法**选取边的过程如下页图所示。



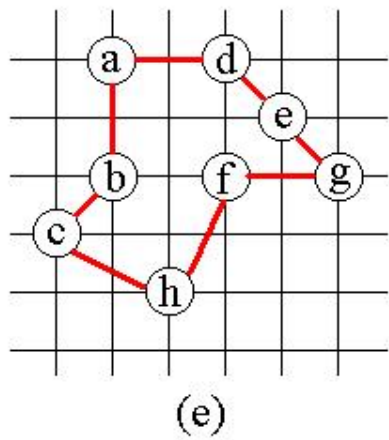
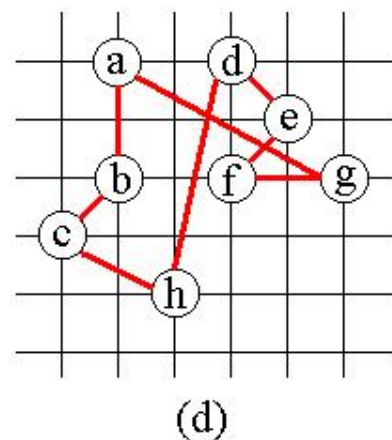
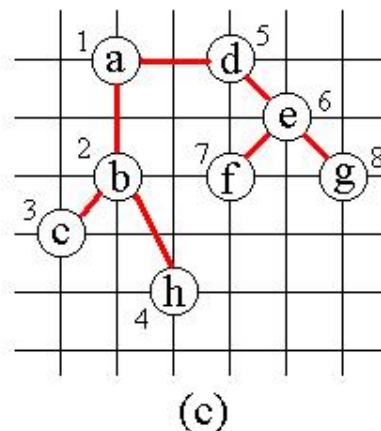
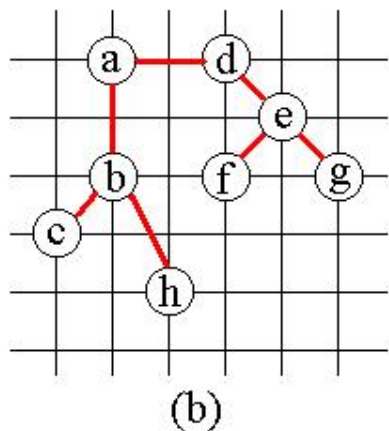
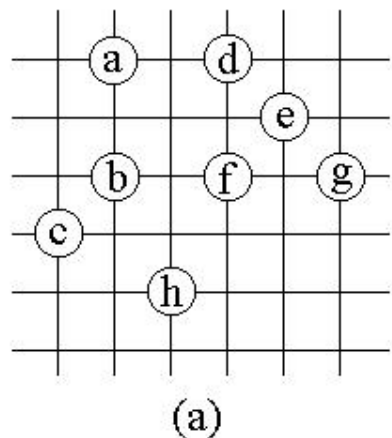


最小生成树





9.3.1 具有三角不等式性质的旅行售货员问题举例



(b) 表示找到的最小生成树T；(c) 表示对T作前序遍历的次序；(d) 表示L产生的哈密顿回路H；(e) 是G的一个最小费用旅行售货员回路。



近似算法性能分析

- 第二步的复杂度为 $O(|V|^2)$,第三四步的复杂度为 $O(|V|)$,因为最小生成树恰有 $|V|-1$ 条边
- 对于近似解 A , 由于 L 是遍历树 T 的每条边两次得到的回路, 则 $|A(L)|=2|T|$, 而 A 是删除某些重复边得到的结果, 因此 $|A|\leq 2|T|$
- 对于最优解 OPT , 删除一条边可得生成树, 且该树的代价一定不低于最小生成树,
 $|OPT|\geq |T|$



9.3.2 一般的旅行售货员问题

在费用函数不一定满足三角不等式的一般情况下，不存在具有常数性能比的解TSP问题的多项式时间近似算法，除非 $P=NP$ 。换句话说，若 $P \neq NP$ ，则对任意常数 $\rho > 1$ ，不存在性能比为 ρ 的解旅行售货员问题的多项式时间近似算法。



9.3.2 一般的旅行售货员问题

- 设 $G_1=(V, E_1)$ 为顶点集 V 上的一个完全图, E_1 中每一条边的费用 $c(u, v)$ 定义为

$$c(u, v) = \begin{cases} 1, & (u, v) \in E \\ p|V| + 1, & (u, v) \in E_1 - E \end{cases}$$

- 如此定义的 G_1 和 c 可根据 G 关于 $|V|$ 和 $|E|$ 的多项式时间构造
- 若 G 有哈密顿回路, 则 c 赋给其每边的费用均为1, G_1 含有一个费用为 $|V|$ 的旅行回路; 若不存在哈密顿回路, 则 G_1 必然用到了不在哈密顿回路 E 中的边, 因此 $(p|V|+1)+(|V|-1) > p|V|$



9.3.2 一般的旅行售货员问题

- 用近似算法A解旅行售货员问题 (G, c) , 则求出的近似解 $c(H) \leq \rho c(H^*)$, H^* 为最优解
 - 当G有哈密顿回路时, $c(H) = c(H^*) = |V|$, 由近似算法找到的旅行回路费用为 $c(H) \leq \rho c(H^*) = \rho |V|$, 于是该旅行回路也是一条哈密顿回路
 - 反之, 若近似算法找出的旅行回路 $c(H) > \rho |V|$, 则 $\rho |V| < c(H) \leq \rho c(H^*)$, 于是 $c(H^*) > |V|$, 即旅行售货员最优回路 H^* 费用 $> |V|$, 故不存在哈密顿回路
- 用近似算法求出H之后, 只需要再判断其费用是否为 $|V|$, 即可判断是否存在一条哈密顿回路
- A可在多项式时间内完成, 在 $P \neq NP$ 下不可能



9.4 集合覆盖问题的近似算法

问题描述：给定全集 X ，以及 n 个子集 $S_1, \dots, S_k$ ，且 $\bigcup S_i = X$ ，每个集合的费用为 $c_1, \dots, c_k$ ，找到子集合 $S_1, \dots, S_d$ ，最小化总费用 $c_1 + \dots + c_d$ ，并满足 $\bigcup_{i=1}^d S_i$

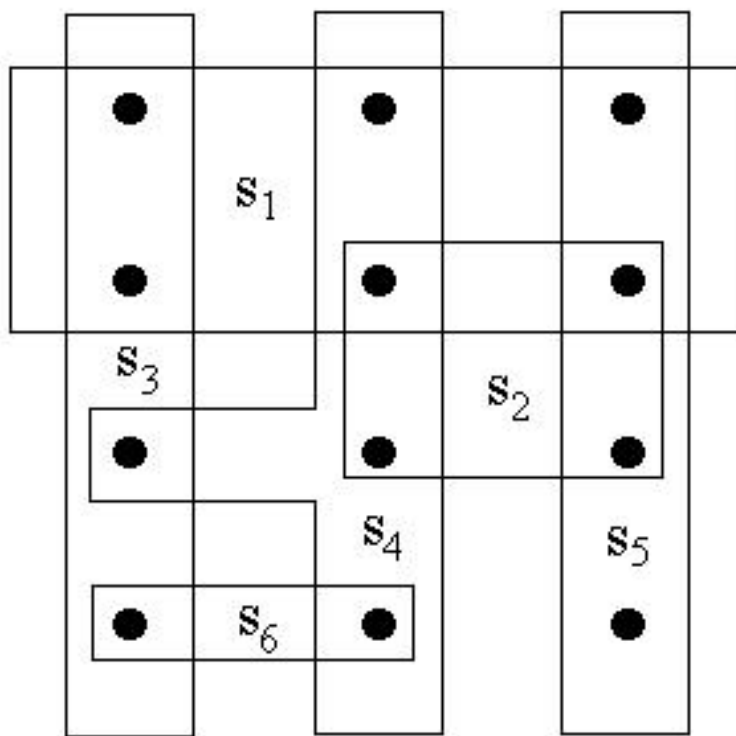
集合覆盖问题的一个实例 $\langle X, F \rangle$ 由一个有限集 X 及 X 的一个子集族 F 组成。子集族 F 覆盖了有限集 X 。也就是说 X 中每一元素至少属于 F 中的一个子集，即 $X = \bigcup_{S \in F} S$ 。对于 F 中的一个子集 $C \subseteq F$ ，若 C 中的 X 的子集覆盖了 X ，即 $X = \bigcup_{S \in C} S$ ，则称 C 覆盖了 X 。集合覆盖问题就是要找出 F 中覆盖 X 的最小子集 C^* ，使得

$$|C^*| = \min \{ |C| \mid C \subseteq F \text{ 且 } C \text{ 覆盖 } X \}$$



9.4 集合覆盖问题的近似算法

集合覆盖问题举例：



用12个黑点表示集合 X 。
 $F = \{S_1, S_2, S_3, S_4, S_5, S_6\}$ ，如图所示。
容易看出，对于这个例子，最小集合覆盖为：
 $C = \{S_3, S_4, S_5\}$ 。



9.4 集合覆盖问题的近似算法

集合覆盖问题近似算法——贪心算法

```
Set greedySetCover (X, F)
{
    U=X;
    C=∅;
    while (U !=∅) {
        选择F中使 $|S \cap U|$ 最大的子集S;
        U=U-S;
        C=C ∪ {S};
    }
    return C;
}
```

算法的循环体最多执行 $\min\{|X|, |F|\}$ 次。而循环体内的计算显然可在 $O(|X| |F|)$ 时间内完成。因此，算法的计算时间为 $O(|X| |F| \min\{|X|, |F|\})$ 。由此即知，该算法是一个多项式时间算法。



集合覆盖问题

- 对于图例，算法一次选出 S_1, S_4, S_5, S_3 ，最多执行循环 $\min(|X|, |F|)$ 次，循环内计算复杂度为 $O(|X||F|)$ ，于是该算法可在多项式时间内求解
- 在算法将 S_i 加入子集族 C 时，给 S_i 一个费用1，并平摊到 $S_i - \bigcup_{j \leq i-1} S_j$ 中的元素，于是算法终止时得到子集族 C ，其总费用为 $|C| = \sum_{x \in X} C_x$
- $|C| \leq |C^*| H(\max_{S \in F} \{|S|\})$ ，于是ratio bound为 $H(\max_{S \in F} \{|S|\})$



- 这是由于 $|C| = \sum_{x \in X} C_x \leq \sum_{S \in C^*} \sum_{x \in S} C_x$

利用 $\sum_{x \in S} C_x \leq H(|S|)$

- 于是

$$\begin{aligned} |C| &\leq \sum_{S \in C^*} H(|S|) \\ &\leq |C^*| H(\max_{S \in F} \{|S|\}) \end{aligned}$$



- 要证明 $\sum_{x \in S} C_x \leq H(|S|)$, 设 $u_i = |S - \bigcup_{j \leq i} S_j|$ 表示算法选择 $S_1, \dots, S_i$ 之后 S 中尚存的未被覆盖的元素个数, 其中 $u_0 = |S|$, 设 k 为数列 $u_0, u_1, \dots$ 中第一个等于 0 的下标。 S 被 $S_1 \dots S_k$ 所覆盖
- S 中有 $u_{i-1} - u_i$ 个元素被 S_i 第一次覆盖, 于是

$$\sum_{x \in S} C_x = \sum_{i=1}^k \frac{u_{i-1} - u_i}{|S_i - \bigcup_{j \leq i-1} S_j|}$$

跟据贪心算法性质, S 所覆盖的新元素不会比 S_i 多, 否则算法选择 S , 于是

$$\left| S_i - \bigcup_{j \leq i-1} S_j \right| \geq \left| S - \bigcup_{j \leq i-1} S_j \right| = u_{i-1}$$



■ 由此可得

$$\sum_{x \in S} C_x = \sum_{i=1}^k \frac{u_{i-1} - u_i}{|S_i - \bigcup_{j \leq i-1} S_j|} \leq \sum_{i=1}^k \frac{u_{i-1} - u_i}{u_{i-1}}$$

利用任一正整数有 $a < b$, 有 $H(b) - H(a) = \sum_{i=a+1}^b \frac{1}{i} \geq (b - a)/b$

得到

$$\sum_{x \in S} C_x \leq \sum_{i=1}^k \frac{u_{i-1} - u_i}{u_{i-1}} \leq \sum_{i=1}^k H(u_{i-1}) - H(u_i)$$

$$= H(u_0) - H(u_k) = H(u_0) = H(|S|)$$



9.5 子集合问题的近似算法

问题描述：设子集和问题的一个实例为 $\langle S, t \rangle$ 。其中， $S = \{x_1, x_2, \dots, x_n\}$ 是一个正整数的集合， t 是一个正整数。子集和问题判定是否存在 S 的一个子集 S_1 ，使得 $\sum_{x \in S_1} x = t$ 。



9.5.1 子集合问题的指数时间算法

```
int exactSubsetSum (S, t)
{
    int n=|S|;
    L[0]={0};
    for (int i=1; i<=n; i++) {
        L[i]=mergeLists(L[i-1], L[i-1]+S[i]);
        删去L[i]中超过t的元素;
    }
    return max(L[n]);
}
```

算法以集合 $S=\{x_1, x_2, \dots, x_n\}$ 和目标值 t 作为输入。算法中用到将2个有序表 $L1$ 和 $L2$ 合并成为一个新的有序表的算法
`mergeLists(L1, L2)`。



子集合问题的指数时间算法

- 用 P_i 表示 $\{x_1, \dots, x_i\}$ 的所有可能的子集和 $P_0 = \{0\}$, 有 $P_i = P_{i-1} \cup (P_{i-1} + x_i)$
 - $S = \{1, 4, 5\}, P_0 = \{0\}, P_1 = \{0, 1\}, P_2 = \{0, 1, 4, 5\}, P_3 = \{0, 1, 4, 5, 6, 9, 10\}$
- 算法得到的表 $L[i]$ 是一个包含了 P_i 中所有不超过 t 的元素的有序表
 - $|P_i| = 2^i$, 最坏情况下 $|L(i)| = |P_i| = 2^i$



9.5.2 子集合问题的完全多项式时间近似格式

基于算法exactSubsetSum，通过对表 $L[i]$ 作适当的修整建立一个子集和问题的完全多项式时间近似格式。

在对表 $L[i]$ 进行修整时，用到一个修整参数 δ ， $0 < \delta < 1$ 。用参数 δ 修整一个表 L 是指从 L 中删去尽可能多的元素，使得每一个从 L 中删去的元素 y ，都有一个修整后的表 L_1 中的元素 z 满足 $(1 - \delta)y \leq z \leq y$ 。可以将 z 看作是被删去元素 y 在修整后的新表 L_1 中的代表。

举例：若 $\delta = 0.1$ ，且 $L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle$ ，则用 δ 对 L 进行修整后得到 $L_1 = \langle 10, 12, 15, 20, 23, 29 \rangle$ 。其中被删去的数11由10来代表，21和22由20来代表，24由23来代表。



9.5.2 子集合问题的完全多项式时间近似格式

对有序表L修整算法

```
List trim(L,  $\delta$ )
{
    int m=|L|;
    L1=  $\langle L[1] \rangle$  ;
    int last=L[1];
    for (int i=2; i<=m; i++) {
        if (last<(1- $\delta$ )*L[i]) {
            将L[i]加入表L1的尾部;
            last=L[i];
        }
    }
    return L1;
}
```

子集和问题近似格式

```
int approxSubsetSum(S, t,  $\epsilon$ )
{
    n=|S|;
    L[0]=  $\langle 0 \rangle$  ;
    for (int i=1; i<=n; i++) {
        L[i]=Merge-Lists(L[i-1],
                        L[i-1]+S[i]);
        L[i]=Trim(L[i],  $\epsilon/n$ );
        删去L[i]中超过t的元素;
    }
    return max(L[n]);
}
```



子集合问题的完全多项式 时间近似格式

- 算法Trim的时间复杂度为 $O(m)$
- 对于 $S=\{104,102,201,101\}$, $t=308$, $\epsilon=0.2$, 修整参数为 $0.2/4=0.05$, 每次经过合并、修整、删除三个阶段
 - $L[1]=\{0,104\}$, $L[1]=\{0,104\}$, $L[1]=\{0,104\}$
 - $L[2]=\{0,102,104,206\}$, $L[2]=\{0,102,206\}$, $L[2]=\{0,102,206\}$
- 算法返回近似解302, 容易得出该例的最优解为 $104+102+101=307$



近似算法性能分析

- 算法计算出的近似解是 S 的子集和，其关于最优解的相对误差不超过预先设定的 ϵ
- 算法是子集和问题的一个多项式时间近似，即复杂度是关于 n 和 $1/\epsilon$ 的多项式
 - 对于 P_i 中任一不超过 t 的元素 y ， $L[i]$ 中有一 x 使得 $(1-\epsilon/n)^i y \leq x \leq y$ ，对于最优值 c^* 存在于 P_n ，因此 $L[n]$ 中有一 x 使得 $(1-\epsilon/n)^n c^* \leq x \leq c^*$
 - 由于算法返回 $L[n]$ 中最大元素 z ， $x \leq z \leq c^*$ ，于是 $(1-\epsilon/n)^n c^* \leq z \leq c^*$ ，利用 $(1-\epsilon) \leq (1-\epsilon/n)^n$ ，得到 $(1-\epsilon)c^* \leq z \leq c^*$ ，进而 $(c^*-z)/c^* = 1-z/c^* \leq \epsilon$



近似算法性能分析

- 每次对有序表 $L[i]$ 做合并、修整、删除的计算时间为 $O(|L[i]|)$,于是整个算法计算时间不超过 $O(n|L[n]|)$
- 对 $L[i]$ 修整后,元素相对间距满足 $a/b > 1/(1 - \epsilon/n)$,而表中最大元素不超过 t ,于是元素个数不超过 $\ln t / \ln(1/(1 - \epsilon/n)) = \ln t / (-\ln(1 - \epsilon/n)) \leq t/(\epsilon/n)$
- 于是整个算法复杂度为 $O(n^2/\epsilon)$